Tema 2: Colecciones

Pablo Gordillo (pabgordi@ucm.es)

Departamento de Sistemas Informáticos y Computación

Facultad de Informática

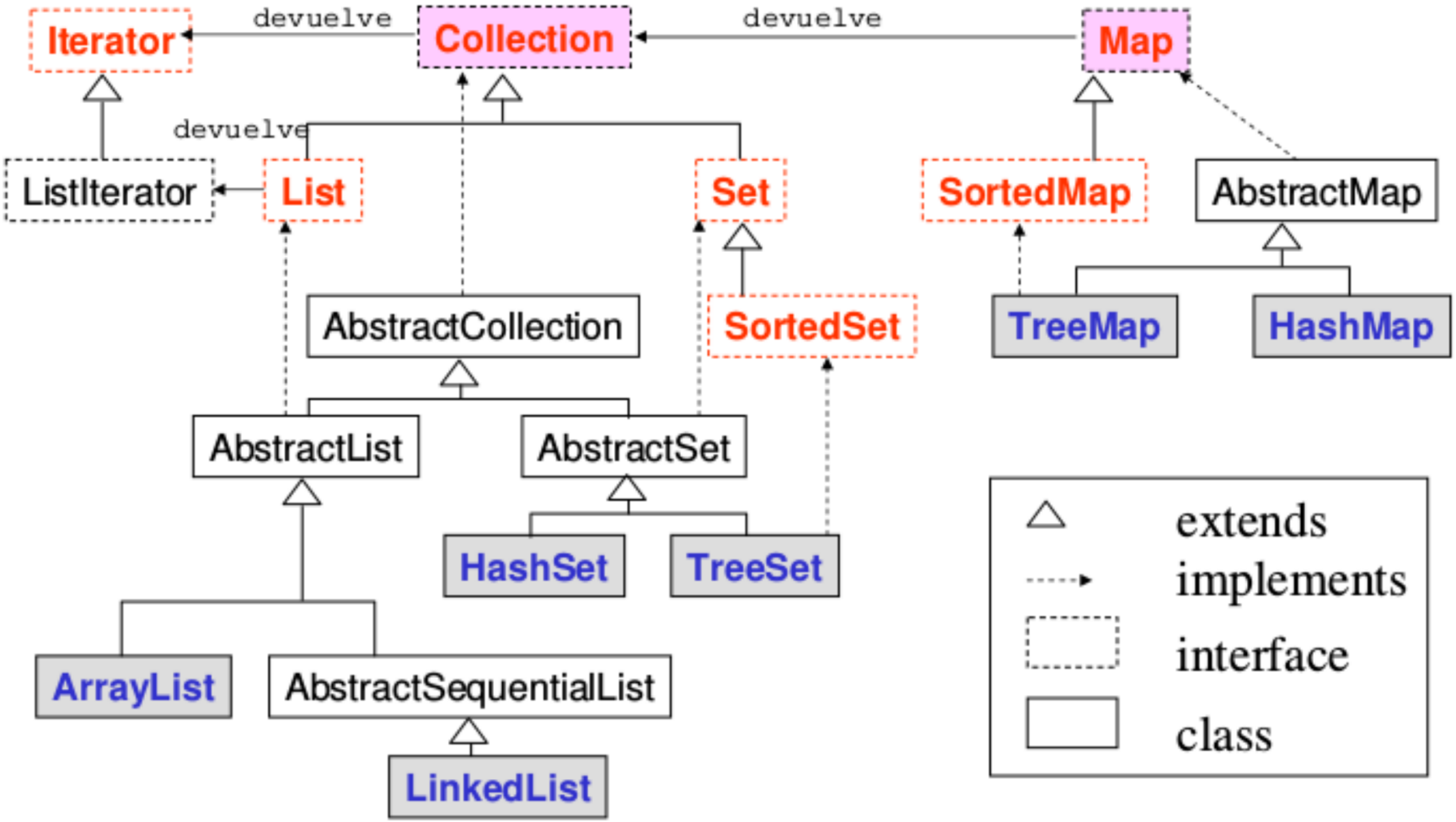


(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

Colecciones en Java

- Es un Object que agrupa varios elementos en un solo objeto (lista, map, conjunto,pila, ...).
- Es una construcción que permite agrupar elementos del mismo tipo, consultarlos, manipularlos, buscarlos, etc.
- Normalmente representan elementos del mismo tipo (list of Alumnos, String, etc).
- Todas las colecciones en Java están implementadas como **clases genéricas (List<T>, ArrayList<T>, etc.).**
- Están definidas en **java.util**.
- Las operaciones principales de una colección son:
  - **add(T e):** añade un elemento a la colección.
  - **Iterator():** devuelve un iterador que permite recorrer la colección, visitando cada elemento una vez.
  - **size():** devuelve el número de elementos de la colección.
  - **contains(T e):** comprueba si un elemento pertenece a la colección.

Colecciones en Java



Antes de Empezar...

Hay algunos "convenios" que se deben tener en cuenta al utilizar o desarrollar colecciones.

1. El método **equal** es el que se usa para comprobar si dos objetos son iguales: **o1.equal(o2)**.
2. La interfaz java.util.Comparable se usa para determinar el orden entre dos elementos. Si no está implementada se puede usar java.util.Comparator.

```
public interface Comparable<T> {
  public int compareTo(T e);
}

public interface Comparator<T> {
  public int compare(T e1, T e2);
}
```

3. El método hashCode() se usa para calcular el índice del "bucket" en las tablas hash, etc.

La interfaz Collection<T>

Es la interfaz que define las operaciones que serán comunes a todas las colecciones de Java.

Existen dos tipos báscios de colecciones (extienden a Collection<T>), que son:

- Listas: definidas a través de la interfaz **List<T>**.
- Conjuntos: definidos a través de la interfaz **Set<T>**.

La interfaz Collection<T>

- **Operaciones básicas de consulta:**
  - size(): devuelve el tamaño de la colección.
  - isEmpty(): comprueba si la colección está vacía.
  - contains(Object e): Comprueba si el objeto **e** pertenece a la colección. Utiliza el método **equals**. Es conveniente definir este método en la clase del objeto **e**.
- **Operaciones básicas de modificación:**
  - add(T e): Añade un elemento a la colección.
  - remove(Object e): elimina **e** de la colección usando **equals** para encontrarlo. Devuelve un booleano para indicar si se ha eliminado.
  - clear(): elimina todos los elementos de la colección.
  - addAll(Collection<? extends T> col): añade todos los elementos de **col** a la colección.
  - removeAll(Collection<?> col): elimina todos los objetos contenidos en **col**.

La interfaz Collection<T>

- Las operaciones **remove** y **contains** llevan un parámetro de tipo **Object**. Sin embargo, el método **add** lo lleva de tipo **T**.
- El motivo es que **se permite que se pueda eliminar o buscar un elemento a través de alguna de sus componentes**. Sin embargo como las colecciones están obligadas a tener elementos del mismo tipo, solo se permite añadirlos de tipo **T**.

```
public interface Collection <T> extends Iterable<T> {
  int size ();
  boolean isEmpty();
  boolean contains (Object o);
  boolean add(T e);
  boolean remove(Object o);
  boolean containsAll (Collection <?> c);
  boolean addAll(Collection <? extends T> c);
  boolean removeAll(Collection <?> c);
  void clear ();
  ...
}
```

La interfaz Collection<T>

- Las operaciones **remove** y **contains** llevan un parámetro de tipo **Object**. Sin embargo, el método **add** lo lleva de tipo **T**.
- El motivo es que **se permite que se pueda eliminar o buscar un elemento a través de alguna de sus componentes**. Sin embargo como las colecciones están obligadas a tener elementos del mismo tipo, solo se permite añadirlos de tipo **T**.

```
public interface Collection <T> extends Iterable<T> {
  int size ();
  boolean isEmpty();
  boolean contains (Object o);
  boolean add(T e);
  boolean remove(Object o);
  boolean containsAll (Collection <?> c);
  boolean addAll(Collection <? extends T> c);
  boolean removeAll(Collection <?> c);
  void clear ();
  ...
}
```

- Iterable nos permite recorrer la colección (lo veremos después).

La interfaz List<T>

- Representa una sucesión de elementos que **puede contener repeticiones**.
- Se puede acceder a un elemento a través de su posición en la lista.
- Las posiciones van de **0** a **size()-1**. Si se accede a una posición ilegal, se genera una excepcion **IndexOutOfBoundsException**.
- Además de las operaciones de **Collections** tiene las siguientes:
  T get(int index)
  T set(int index, T elem)
  void add(int index, T elem)
  T remove(int index)

La interfaz List<T>

```
public interface List <T> extends Collection<T> {
  T get(int index);
  T set(int index, T element);
  void add(int index, T element);
  T remove(int index);
  int indexOf(Object o); //se puede buscar por alguna
  int lastIndexOf (Object o); // clave
  List <T> subList(int fromIndex, int toIndex);
  ...
}
```

Clases que implementan List<T>

- ArrayList<T>: Implementación basada en arrays redimensionables.
  - Operaciones de inserción y modificación ineficientes.
  - Operaciones de creación y consulta rápidas.

- LinkedList<T>: implementación basada en listas doblemente enlazadas.
  - Inserción y modificación rápidas, sobre todo al inicio o al final de la lista: **addFirst, addLast, removeFirst, removeLast**
  - Acceso aleatorio a elementos ineficiente.
  - Acceso eficiente al inicio y al final: **T getFirst(), T getLast()**

Clases que implementan List<T>

- Vector<T>: Similar a **ArrayList**.
  - Solo una hebra puede manipular un **Vector** (sincronizado), mientras que un **ArrayList** puede ser manipulado simultáneamente por varias hebras (no sincronizado).
  - **Vector** redimensiona al doble de su tamaño mientras que **ArrayList** solo a la mitad.
  - La clase Vector se añadió con posterioridad a las colecciones. *Solo conviene usarlos cuando en concurrencia no queremos que varias hebras accedan a la colección*.
- Stack<T>: Hereda de **Vector<T>** y ofrece la funcionalidad propia de una pila (push(), pop(), etc.).

Interfaz Set<T>

- Implementa las operaciones de conjuntos y por tanto **no admite repetición de elementos**.
- La clase **HashSet<T>** implementa la interfaz **Set<T>**. **HashSet<T>** guarda los elementos del conjunto en una *tabla hash*. Los elementos repetidos se evitan comparando los hashCode de los elementos.
- La interfzas **SortedSet<T>** extiende a la interfaz **Set<T>**. Es similar pero los elementos se mantienen internamente ordenados. Es implementada por la clase **TreeSet<T>**, que implementa los conjuntos ordenados basándose en *árboles binarios balanceados*. Para poder utilizarlos hay que definir un orden.
- Las operaciones de añadir y modificar son más lentas en **TreeSet** que en **HashSet**. Las búsquedas son más rápidas en estructuras ordenadas. En general, es preferible usar un **HashSet** y posteriormente ordenarlo y transformarlo en un **TreeSet**.

Interfaz Set<T>

- Por defecto, en **TreeSet<T>**, los iteradores usan orden ascendente, pero se puede obtener uno que usa orden descendente mediante el método **TreeSet.descendingIterator()**.
- **LinkedHashSet** es como **HashSet**, pero garantiza el orden de inserción al iterar los elementos.
- **EnumSet** es un conjunto de objetos de tipo **Enum<T>**. Es una implementación eficiente y compacta ya que sabemos que elementos puede contener.
- **NavigableSet** es un subtipo de **SortedSet**. Se comporta igual, pero tiene otros métodos de "navegación".

La interfaz Map<K,V>

- Interfaz para representar pares <**clave,valor**>.
- Las claves son únicas y pueden ser no numéricas. Cada clave puede contener un valor como máximo.
- No es un subtipo de Collection. Por tanto, se comporta de manera un poco diferente.
- Contiene una interfaz estática anidada **static interface Map.Entry<K,V>**
- Su implementación contiene distintas colecciones:
  - Conjunto de claves (**Set<K>**)
  - Colección de valores (**Collection<V>**)
  - Conjunto de pares <**clave,valor**> (**Map.Entry<K,V>**)
- Una implementación de **Map<K,V>** es **HashMap<K,V>**, cuya implementación está basada en una tabla hash.
- La interfaz **Map<K,V>** es extendida por la interfaz **SortedMap<K,V>**, que mantiene los elementos orenados por la clave. **SortedMap<K,V>** es implementada por **TreeMap<K,V>**, que basa su implementación en árboles balanceados. El conjunto de claves en este caso es un **SortedSet<K>**.

La interfaz Map<K,V>

En los métodos de la interfaz tenemos:

- **V put(K clave, V valor)**: Inserta el par (clave,valor) en el mapa y devuelve el valor que hubiera en esa clave en caso de haberlo.
- **V get(K clave)**: Devuelve el valor asociado a la clave. Si no existe la clave devuelve null.
- **Set<K> KeySet()**: Devuelve el conjunto de claves.
- **Collection<V> values()**: Devuelve la colección de valores.
- **boolean containsKey(K clave)**: Indica si existe una clave.
- **Set<Map.Entry<K,V>> entrySet()**: Devuelve el conjunto de todos los pares **Map.Entry<K,V>**.

La interfaz Map<K,V>

```
public interface Map<K,V> {
  int size();
  boolean isEmpty();
  boolean containsKey(Object key);
  boolean containsValue(Object value);
  V get(Object key);
  V put(K key, V value);
  V remove(Object key);
  void putAll(Map<? extends K, ? extends V> m);
  void clear();
  Set<K> keySet();
  Collection <V> values();
  Set<Map.Entry<K, V>> entrySet();
  ...
}
```

Los Iteradores

- En Java **las colecciones son iterables**, es decir, que se pueden recorrer sus elementos.
- Java proporciona la interfaz **Iterable<T>**, que es implementada por aquellas clases sobre las que se puede iterar, en particular las colecciones (pero no solo ellas). También es posible recorrer una colección usando "for each".

```
public interface Iterable <T> {
  Iterator <T> iterator();
}
```

```
public interface Iterator <T> {
  boolean hasNext(); // quedan elementos en la colección?
  T next();          // avanza iterador y devuelve el elemento
  void remove();         // elimina el elemento donde está el
}                    // iterador, y el iterador retrocede
                     // una posición
```

Recorriendo una Colección

- Se puede recorrer usando "for each".

```
public class Recorrido {
  public static void eliminaPares(List<Integer> lista) {
    List<Integer> listaPares = new LinkedList<Integer>();
    for (Integer i : lista)
    if (i % 2 == 0) listaPares.add(i);
    lista.removeAll(listaPares);
  }

  public static void main(String[] args) {
    List<Integer> lista = new LinkedList<Integer>();
    lista.add(4);
    lista.add(3);
    lista.add(2);
    lista.add(1);
    System.out.println(lista);
    eliminaPares(lista);
    System.out.println(lista);
  }
}
```

Recorriendo una Colección

- **Con Iterator:**.

```
public class Recorrido {
  public static void eliminaPares(List<Integer> lista) {
    List<Integer> listaPares = new LinkedList<Integer>();
    Iterator<Integer> it = lista.iterator();
    while (it.hasNext()) {
      Integer n = it.next();
      if (n % 2 == 0) listaPares.add(n);
    lista.removeAll(listaPares);
  }

  public static void main(String[] args) {
    List<Integer> lista = new LinkedList<Integer>();
    lista.add(4);
    lista.add(3);
    lista.add(2);
    lista.add(1);
    System.out.println(lista);
    eliminaPares(lista);
    System.out.println(lista);
  }
}
```

Recorriendo una Colección

- **Qué diferencia hay entre ambas funciones?**

```
public class Recorrido {
  public static void eliminaPares(List<Integer> lista) {
    List<Integer> listaPares = new LinkedList<Integer>();
    Iterator<Integer> it = lista.iterator();
    while (it.hasNext()) {
      Integer n = it.next();
      if (n % 2 == 0) listaPares.add(n);
    lista.removeAll(listaPares);
  }
}
...
}
```

```
public class Recorrido {
  public static void eliminaPares2(List<Integer> lista) {
    List<Integer> listaPares = new LinkedList<Integer>();
    Iterator<Integer> it = lista.iterator();
    while (it.hasNext()) {
      if (it.next() % 2 == 0) listaPares.add(it.next());
      lista.removeAll(listaPares);
    }
    ...
  }
}
```

Colecciones Ordenadas

- Recuerda que las colecciones ordenadas (**SortedSet, SortedMap**) necesitan definir su propio orden. Para ello, pueden implementar la interfaz **Comparable<T>**, o bien utilizar la interfaz **Comparator<T>** en la constructora.

```
public interface Comparable<T> {
  public int compareTo(T e);
}

public interface Comparator<T> {
  public int compare(T e1, T e2);
}
```

- **o.compareTo(T e)** devuelve 0 si los elementos son iguales, -1 si **o** es menor que **e** y 1 si **o** es mayor que e. El método **compare(T e)** se comporta de forma análoga.

Colecciones Ordenadas

```
public class Cuenta implements Comparable<Cuenta> {
  private int numCuenta;
  private int saldo;

  public Cuenta(int nc, int s) {
    this.numCuenta = nc;
    this.saldo = s;
  }

  @Override
  public int compareTo(Cuenta cuenta) {
    if (this.numCuenta == cuenta.numCuenta) return 0;
    else if (this.numCuenta < cuenta.numCuenta) return -1;
    else return 1;
  }

  public String toString() {
    return "Numero cuenta: " + this.numCuenta + "\n"
    + "Saldo: " + this.saldo;
  }
```

Colecciones Ordenadas

```
public class Cliente {
  private int nif;

  //no se fija orden en la constructora
  private TreeSet<Cuenta> cuentas = new TreeSet<Cuenta>();

  ...

  // se añade teniendo en cuenta el orden definido en Cuenta
  public boolean addCuenta(Cuenta c){ return cuentas.add(c); }

  public String toString() {
    Iterator <Cuenta> it = cuentas.iterator();
    String s = "";
    while (it.hasNext()) {
      Cuenta c = it.next();
      s = s + c.toString() + "\n";
    }
    return s;
  }
}
```

Colecciones Ordenadas

```
public static void main(String[] args) {
  Cliente c = new Cliente(2);
  Cuenta cuenta1 = new Cuenta(1,1000);
  c.addCuenta(cuenta1);
  Cuenta cuenta3 = new Cuenta(3,3000);
  c.addCuenta(cuenta3);
  Cuenta cuenta2 = new Cuenta(2,2000);
  c.addCuenta(cuenta2);
  System.out.println (c.toString());
}
```

```
Número de cuenta: 1
Saldo: 1000
Número de cuenta: 2
Saldo: 2000
Número de cuenta: 3
Saldo: 3000
```

Colecciones Ordenadas

Se puede fijar el orden de una colección en su constructora utilizando la interfaz **Comparator<T>**.

```
public class Cliente {
  private int nif;
  private TreeSet<Cuenta> cuentas;
  ...
  //se fija orden en la constructora
  public Cliente (int n, Comparator<Cuenta> comp) {
    this.nif = n;
    cuentas = new TreeSet<Cuenta>(comp);
  }

  public boolean addCuenta(Cuenta c){ return cuentas.add(c); }
}

public class Cuenta { // no implementa a Comparable<T>
  private int numCuenta;
  private int saldo;
  ...
  public int getNumCuenta() { return this.numCuenta; }
  public int getSaldo(){ return this.saldo; }

}
```

Colecciones Ordenadas

```
//ordena por número de cuenta
public class OrdenaPorNumeroCuenta implements Comparator<Cuenta> {

  @Override
  public int compare(Cuenta cuenta1, Cuenta cuenta2) {
    if (cuenta1.getNumCuenta() == cuenta2.getNumCuenta()) return 0;
    else if (cuenta1.getNumCuenta() < cuenta2.getNumCuenta()) return -1;
    else return 1;
  }
}

//ordena por saldo
public class OrdenaPorSaldo implements Comparator<Cuenta> {

  @Override
  public int compare(Cuenta cuenta1, Cuenta cuenta2) {
    if (cuenta1.getSaldo() == cuenta2.getSaldo()) return 0;
    else if (cuenta1.getSaldo() < cuenta2.getSaldo()) return -1;
    else return 1;
  }
}
```

Colecciones Ordenadas

```
//Cada cliente tiene un orden distinto en sus cuentas
Cliente  c = new Cliente(2, new OrdenarPorNumeroCuenta());
Cuenta cuenta1 = new Cuenta(1,1000);
c.addCuenta(cuenta1);
Cuenta cuenta3 = new Cuenta(3,3000);
c.addCuenta(cuenta3);
Cuenta cuenta2 = new Cuenta(2,2000);
c.addCuenta(cuenta2);
System.out.println (c.toString());

Cliente  c1 = new Cliente(2, new OrdenarPorSaldo());
cuenta1 = new Cuenta(1,3000);
c1.addCuenta(cuenta1);
cuenta3 = new Cuenta(3,1000);
c1.addCuenta(cuenta3);
cuenta2 = new Cuenta(2,2000);
c1.addCuenta(cuenta2);
System.out.println (c1.toString());
```

```
Número de cuenta: 1
Saldo: 1000
Número de cuenta: 2
Saldo: 2000
Número de cuenta: 3
Saldo: 3000
```

```
Número de cuenta: 3
Saldo: 1000
Número de cuenta: 2
Saldo: 2000
Número de cuenta: 1
Saldo: 3000
```

Ejemplo 1

```
public class TreeHashMap {
  public static void test1 (Map<String, Integer> m) {
    System.out.println ("-------");
    m.put("Mike", 23);
    m.put("John", 16);
    m.put("Andres", 5);
    System.out.println (m);
  }
}

public static void main(String[] args) {
  test1 (new HashMap<String, Integer>());// estructura sin orden
  test1 (new TreeMap<String, Integer>());// estructura ordenada
}
```

```
- - - - - - -
{Andres=5, Mike=23, John=16}
- - - - - - -
{Andres=5, John=16, Mike=23}
```

En un HasMap (estructura sin orden) los elementos no se muestran en el orden de salida ya que esta estructura no almacena el orden en el cual los elementos se han ido introduciendo.
En el TreeMap, se muestran ordenados por la clave. Al ser la clave de tipo String, se usa por defecto el orden lexicográfico.

Ejemplo 1

```
public class Person {
  private int id;
  private String name;
  public Person(String name, int id){...}
  @Override
  public String toString(){ return name+":"+id;}
}

public static void test2(Map<Person, String> m) {
  System.out.println("------");
  m.put(new Person("Mike", 23), "Prof.");
  m.put(new Person("John", 16), "Student");
  m.put(new Person("Andres", 5), "Admin");
  System.out.println(m);
}
```

```
------
{Mike:23=Prof., John:23=Student, Andres:5=Admin}
```

Ejemplo 1

```
public class Person {
  private int id;
  private String name;
  public Person(String name, int id){...}
  @Override
  public String toString(){ return name+":"+id;}
}

public static void test2(Map<Person, String> m) {
  System.out.println("-------");
  m.put(new Person("Mike", 23), "Prof.");
  m.put(new Person("John", 16), "Student");
  m.put(new Person("Andres", 5), "Admin");
  System.out.println(m);
}

public static void main(String[] args) {
  test2(new HashMap<Person, String>());// estructura sin orden
}
```

```
- - - - - -
{Mike:23=Prof., John:23=Student, Andres:5=Admin}
```

Ejemplo 1

```
public class Person {
  private int id;
  private String name;
  public Person(String name, int id){...}
  @Override
  public String toString(){ return name+":"+id;}
}

public static void test2(Map<String, Integer> m) {
  System.out.println("-------");
  m.put(new Person("Mike", 23), "Prof.");
  m.put(new Person("John", 16), "Student");
  m.put(new Person("Andres", 5), "Admin");
  System.out.println(m);
}

public static void main(String[] args) {
  test2(new TreeMap<Person, String>());// estructura con orden
}
```

Ejemplo 1

```
public class Person {
  private int id;
  private String name;
  public Person(String name, int id){...}
  @Override
  public String toString(){ return name+":"+id;}
}

public static void test2(Map<String, Integer> m) {
  System.out.println("-------");
  m.put(new Person("Mike", 23), "Prof.");
  m.put(new Person("John", 16), "Student");
  m.put(new Person("Andres", 5), "Admin");
  System.out.println(m);
}

public static void main(String[] args) {
  test2(new TreeMap<Person, String>());// estructura con orden
}
```

ERROR!! ¿Cómo ordena? Necesitamos definir un orden en la clase Person!! No existe por defecto.

Ejemplo 1

```
public class Person {
  private int id;
  private String name;
  public Person(String name, int id){...}
  @Override
  public String toString(){ return name+":"+id;}
}

public class Orden implements Comparator<Person> {
  @Override
  public int compare(Person o1, Person o2) {
    return o1.getId() > o2.getId() ? 1 : o1.getId() < o2.getId() ? -1 : 0;
  }
}

public static void test2(Map<String, Integer> m) {
  System.out.println("-------");
  m.put(new Person("Mike", 23), "Prof.");
  m.put(new Person("John", 16), "Student");
  m.put(new Person("Andres", 5), "Admin");
  System.out.println(m);
}
```

Ejemplo 1

```
public class Person {
  private int id;
  private String name;
  public Person(String name, int id){...}
  @Override
  public String toString(){ return name+":"+id;}
}

public class Orden implements Comparator<Person> {
  @Override
  public int compare(Person o1, Person o2) {
    return o1.getId() > o2.getId() ? 1 : o1.getId() < o2.getId() ? -1 : 0;
  }
}

public static void test2(Map<String, Integer> m) {
  System.out.println("-------");
  m.put(new Person("Mike", 23), "Prof.");
  m.put(new Person("John", 16), "Student");
  m.put(new Person("Andres", 5), "Admin");
  System.out.println(m);
}


public static void main(String[] args) {
  test2(new TreeMap<Person, String>(new Orden()));
}
```

Ejemplo 1

Cuál es el problema de este código? Qué mostrará test2?

```
public class Person {
  private int id;
  private String name;
  public Person(String name, int id){...}
  @Override
  public String toString(){ return name+":"+id;}
}

public class Orden implements Comparator<Person> {
  @Override
  public int compare(Person o1, Person o2) {
    return o1.getId() > o2.getId() ? 1 : o1.getId() < o2.getId() ? -1 : 0;
  }
}

public static void test2(Map<String, Integer> m) {
  System.out.println("-------");
  m.put(new Person("Mike", 23), "Prof.");
  m.put(new Person("John", 23), "Student");
  m.put(new Person("Andres", 5), "Admin");
  System.out.println(m);
}

public static void main(String[] args) {
  test2(new TreeMap<Person, String>(new Orden()));
}
```

No se sobreescriben claves repetidas

Ejemplo 1

El método equals y el método hashCode se pueden generar automáticamente en Eclipse

```
public class Person {
  ...

  @Override
  public int hashCode() {
    final int prime = 31;
    int result = 1;
    result = prime * result + id;
    result = prime * result + ((name == null) ? 0 : name.hashCode());
    return result ;
  }

  @Override
  public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null) return false ;
    if (getClass() != obj.getClass()) return false ;
    Person other = (Person) obj;
    if (id != other.id) return false ;
    if (name == null) {
      if (other.name != null) return false ;
    } else if (!name.equals(other.name)) return false ;
    return true;
  }
}
```

Ejemplo 2: Ser Iterable<T>

```
public class MyList<T extends Object> implements Iterable<T> {
  private final static int _INIT_SIZE = 2;
  protected T[] elem;
  protected int last ;

  public MyList() {...}

  public void addElem(T e) {...}

  public T getElem(int index) {...}

  public int size () { return last ; }

  @Override
  public Iterator <T> iterator() {
    return new MyIterator<T>(this);
  }

}
```

MyIterator<T> es una clase interna que nos permite acceder a elem y last

Ejemplo 2: La Implementación del Iterador

```
public class MyIterator<T> implements Iterator<T> {
  private int currentPos = 0;
  private MyList<T> lista;

  public MyIterator(MyList<T> l){ this.lista = l; }

  @Override
  public boolean hasNext() { return currentPos < last; }

  @Override
  public T next() {
    currentPos++;
    return this.lista.elements[currentPos-1];
  }
}
```

Ejemplo 2

```
public static void main(String[] args) {
  MyList<Integer> x = new MyList<Integer>();
  x.addElem(1);
  x.addElem(15);
  x.addElem(5);
  Iterator <Integer> it = x.iterator ();
  while (it.hasNext()) {
    System.out.println (it.next());
  }

  for ( Integer i : x ){
    System.out.println (i); // utiliza internamente el iterador
  }
}
```

Ejemplo 3

Implementar un banco que tiene una **colección de clientes** y un nombre

- Un **cliente** tiene un **NIF** y una **colección de cuentas** ordenadas por el número de cuenta.
- Una **cuenta** tiene un **número de cuenta** y un **saldo**.

```
public class Account {
  private int accountNumber;
  private int money;

  public Account(int ac, int m){...}

  public void addMoney(int m) {this.money = this.money + m}

  public Integer getAccountNumber() {return this.accountNumber;}

  public void removeMoney(int m) {
    if (this.money > m) this.money = this.money - m;
  }

  public int money() { return this.money; }

  public String toString () {
    return "Account info = " + this.accountNumber + ":" + this.money;
  }
}
```

Ejemplo 3

```
public class Client implements Comparable<Client> {
  private int nif;
  private List<Account> accounts;
  // pares (numeroCuenta,Cuenta), para hacer la búsqueda de una
  // cuenta de forma más rápida !!!
  private Map<Integer,Account> mapAccounts;

  public Client(int nif) {
    this.nif = nif;
    this.accounts = new LinkedList<Account>();
    this.mapAccounts = new HashMap<Integer,Account>();
  }

  public int getNif() { return this.nif; }

  public void newAccount(Account ac) {
    this.accounts.add(ac);
    this.mapAccounts.put(ac.getAccountNumber(),ac);
  }

  @Override
  public int compareTo(Client c) {
    if (c.getNif() == this.nif) return 0;
    else if (this.nif < c.getNif()) return -1;
    else return 1;
  }
```

Ejemplo 3

```
public void addMoney(int money, int accountNumber) {
  Account account = mapAccounts.get(accountNumber); // buscamos en el mapa
  if (account!=null) account.addMoney(money); // se actualiza la cuenta
}                                              // en el mapa y en la lista

public void removeMoney(int money, int accountNumber) {
  Account account = mapAccounts.get(accountNumber);
  if (account!=null) account.removeMoney(money);
}
public int totalMoney() { // recorremos usando la lista y no el mapa
  int total = 0;
  for (Account ac: this.accounts) total = total + ac.money();
  return total;
}
public String toString() { // recorremos usando la lista y no el mapa
  String s = "NIF: " + this.nif + "\n";
  for (Account c : this.accounts) s = s + c + "\n";
  return s;
}
public List<Integer> getAccountNumbers() {// recorremos usando la lista y no el mapa
  List<Integer> l = new ArrayList<Integer>();
  for (Account c : this.accounts) l.add(c.getAccountNumber());
  return l;
}
```

Ejemplo 3

```
public class Bank {
  private String name;
  private Set<Client> clients; // ordered by nif
  private Map<Integer,Client> mapClients;
  private static int NEW_ACCOUNT_NUMBER = 1;

  public Bank(String name) {
    this.name = name;
    this.clients = new TreeSet<Client>();
    this.mapClients = new HashMap<Integer,Client>();
  }

  public void addMoney(int money, int nif, int accountNumber){
    Client client = this.mapClients.get(nif);
    if (client != null) client.addMoney(money,accountNumber);
  }

  public void removeMoney(int money, int nif, int accountNumber){
    Client client = this.mapClients.get(nif);
    if (client != null) client.removeMoney(money,accountNumber);
  }

  public void addClient(int nif){ // se añade en el mapa y en la lista
    Client c = new Client(nif);
    if (!clients.add(c)) System.out.println("Error: Ya existe el cliente");
    else this.mapClients.put(nif,c);
  }
```

Ejemplo 3

```
public void newAccount(int nif, int money){
  Client c = this.mapClients.get(nif); // se busca en el mapa
  if (c!=null) {
    Account ac = new Account(Bank.NEW_ACCOUNT_NUMBER,money);
    Bank.NEW_ACCOUNT_NUMBER++;
    c.newAccount(ac);
  }
}

public int totalMoney() { // se recorre el conjunto con iterador
  int total = 0;
  for (Client c: this.clients ) total = total + c.totalMoney();
  return total;
}

public String toString() { // se recorre el conjunto con iterador
  String s = "Bank Name: " + this.name + "\n";
  for (Client c: this.clients ) s = s + c + "\n";
  return s;
}

public List<Integer> getAccountNumbers(int nif){
  Client c = this.mapClients.get(nif); // se busca en el mapa
  return c.getAccountNumbers();
}
```